



AERIS CHECKPOINT 5 — Modular Geometry + Sampling Backbone (Validated)

Use this as your next chat prompt:



CONTEXT — WHAT THIS PROJECT IS

We are building AERIS, a modular AI-driven aircraft design platform.

The system must support:

- Parametric geometry generation (BWB focus now)
- Modular sampling (LHS now, extensible later)
- Modular aero solvers (future step)
- Deterministic dataset generation (critical for ML + certification)
- Clean CLI-driven workflows (no manual hacks)

This is not a script project — it is a scalable architecture for AI-driven design + certification pipelines.



CURRENT STATE (END OF CHECKPOINT 5)

1. Geometry System — STABLE

We implemented:

- GeometryGenerator abstraction
- Registry-based generator selection
- Active generator:
 - `bwb_segmented_v1`

Pipeline:

`config` → `generator` → `sample` → `geometry case` → `artifacts`

Key property:

- ✓ deterministic given sample
 - ✓ generator isolated per family/version
-

2. Sampling System — NOW MODULAR

We transitioned from:

 Hardcoded LHS inside generator

 to

 Modular sampling architecture:

```
aeris/dataset/sampling/  
  base.py  
  registry.py  
  resolver.py  
  samplers/  
    lhs_v1.py
```

Now:

- Sampler selected independently from geometry

- Example:

```
Sampler: lhs_v1
```

```
Seed: 123
```

✓ decoupled from generator

✓ future-ready (Sobol, random, adaptive, active learning)

3. Dataset Pipeline — VERIFIED

Full pipeline tested:

```
config  
  → sampler  
  → generator  
  → geometry cases
```

- dataset
- metadata

Validated:

- deterministic LHS behavior
- reproducibility across runs
- metadata regeneration consistency
- geometry success rate (no silent failures)



ALL CHECKS PASSED

4. Script Architecture — CLEANED

We reorganized:

```
scripts/  
├─ generic/           # architecture + dataset checks  
├─ bwb_segmented_v1/  # generator-specific tools  
└─ utilities/         # helpers
```

Fix applied:



All scripts now resolve project root using:

```
Path(__file__).resolve().parents[2]
```

→ No more broken paths after restructuring

5. API CONSISTENCY — FIXED

Critical architectural fix:



OLD:

```
generate_geometry_case_from_sample(..., rng=...)
```



NEW:

```
sample = sample_bwb_design(...)  
generate_geometry_case_from_sample(..., sample=sample)
```

Meaning:



Sampling and geometry are now cleanly separated

6. Baseline Geometry — VERIFIED

We:

- fixed YAML issues
- enforced valid bounds
- generated baseline geometry
- visualized successfully

Artifacts:

section_3d.csv
planform.png



geometry pipeline is operational



WHAT IS NOW TRUE (IMPORTANT)

This is the real outcome of this checkpoint:



You now have true modularity:

Component	Status	Coupling
Geometry		independent
Sampling		independent
Dataset		orchestrator
CLI		entry point



Determinism is enforced at system level

- seed controls sampling ONLY
- geometry is pure function of sample



This is exactly what you need for ML + certification



Architecture is extensible without rewrite

You can now add:

- new generators → no pipeline change
 - new samplers → no geometry change
 - new solvers → plug later
-



WHAT IS STILL NOT CLEAN (BE HONEST)

This is where most people lie to themselves. Don't.



1. Duplicate logic still exists

You still have:

```
src/aeris/geometry/*  
src/aeris/generators/bwb_segmented_v1/*
```

Some overlap exists (legacy vs new system).



Needs consolidation later.



2. Scripts are still semi-manual

- Scripts exist outside CLI
- Some logic duplicated between CLI and scripts



Eventually:

ALL execution → CLI only
scripts → debugging only



3. Plotting is not standardized

- planform plot logic lives inside generator
- no unified visualization interface



This will break later (especially for aero results)



4. No formal schema validation (yet)

- YAML parsing works
- validation exists
- but no strict schema layer



Future risk: silent config errors



WHAT SHOULD BE TESTED NEXT

Before moving forward:

1. Sampling plug-in test

Add dummy sampler:

random_v1

Verify:

- pipeline works unchanged
 - dataset generation still valid
-

2. Multi-generator readiness

Even with one generator:

Test:

switch generator ID

→ pipeline should not break

3. Failure robustness

Test:

- invalid bounds
- invalid config
- generator failure case



Ensure system fails cleanly (not silently)

4. Large dataset run (stress test)

Run:

n = 500+

Check:

- speed
 - memory
 - logging clarity
-



NEXT PHASE — AERO ARCHITECTURE (CHECKPOINT 6)

Now comes the real difficulty.

You will design:

Aero modular system (same philosophy)

```
aeris/aero/  
  base.py  
  registry.py  
  solvers/
```

aerosandbox_avl.py

Key requirement:



EXACT SAME PATTERN AS GEOMETRY + SAMPLING

Target pipeline

geometry → aero solver → results → dataset

Critical design decisions (must be solved)

1. Input format to solver

- mesh?
- sections?
- aerosandbox object?

2. Output standardization

- CL, CD, Cm
- Cp distributions (future ML target)

3. Failure handling

- AVL crashes
- convergence issues

4. Determinism

- same geometry → same aero results
-



FINAL GOAL OF NEXT CHAT

In the next step, I want:

1. Aero system architecture design

- base class
- registry
- interface definition

2. First solver integration

- aerosandbox AVL wrapper

3. Clean pipeline integration

- geometry → aero → dataset

4. NO hacks

- no direct calls
 - no script shortcuts
 - everything modular
-



FINAL INSTRUCTION

Continue as a system architect, not a coder.

Reject:

- shortcuts
- hidden coupling
- “just make it work”

Enforce:

- modularity
 - determinism
 - extensibility
-

If you want, in the next chat I'll:



design the aero system properly (no rookie mistakes)



and align it with your PhD + startup strategy (this matters a lot)